

Dunaújváros Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

1. A projekt bemutatása és az ötlet leírása.....	1
2. A kód felépítése és működésének magyarázata.....	2
3. A teljes forráskód.....	3
4. A program futtatása és eredményei	11
5. Önreflexió és tapasztalatok	12

Tantárgy neve: Programozás alapjai

Név: Tóth Levente

Osztály: 11.B

Projekt címe: Iskolai versenysorozat szimulációja és adatainak kiértékelése

1. A projekt bemutatása és az ötlet leírása

A portfólió célja egy összetett, C# nyelvű konzolos alkalmazás bemutatása, amely egy 8 fordulóból álló, 10 diák részvételével zajló versenysorozatot szimulál.

Az ötlet leírása: A téma kiválasztásakor az volt a fő szempontom, hogy egy valós életből vett, életszerű problémát modellezzek, amelyben viszonylag sok adatot kell egyszerre kezelni, átlátni és logikailag kiértékelni. Egy iskolai versenysorozat (legyen az tanulmányi vagy sportbajnokság) tökéletes alapnak bizonyult. Itt ugyanis nemcsak egyszerű pontszámokat kell eltárolni, hanem különböző szabályrendszerek alapján rangsorokat kell felállítani, statisztikákat kell készíteni, figyelembe kell venni a holtversenyeket és a kiesési feltételeket (pl. minimális pontszám elérése). A projekt remek lehetőséget biztosított arra, hogy a C# nyelv haladóbb adatszerkezeteit és a modern LINQ lekérdezéseket a gyakorlatban is alkalmazzam.

2. A kód felépítése és működésének magyarázata

A program logikája több, jól elkülöníthető részre bomlik, amelyek a feladat egy-egy pontját hajtják végre:

- **Adatgenerálás és tárolás:** A diákok neveit egy egyszerű `List<string>` tárolja. A versenyek eredményeit egy összetett adatszerkezetben, egy listából álló listában (`List<List<(string, int)>>`) mentjük el. Minden belső lista egy verseny, ami (diák neve, pontszám) adatpárokat (Tuple) tartalmaz. A pontszámokat a `Random` osztály generálja 0 és 100 között.
- **a. Egyéni versenyek rangsora:** A program a versenyeken egy `for` ciklussal megy végig, és az `OrderByDescending` LINQ módszerrel rendezi a diákokat pontszám szerint csökkenő sorrendbe.
- **b - c. Összetett versenyben értékelendő tanulók:** Mivel a szimulációban jelenleg mindenki elindul minden versenyen, a program megvizsgálja, mely listák hossza pontosan 10 fő. A `Distinct()` függvény gondoskodik arról, hogy a nevek kigyűjtésénél ne legyenek duplikációk.
- **d. Összetett verseny rangsora:** A diákok összesített pontszámainak kiszámításához egy szótárat (`Dictionary<string, int>`) alkalmaz a kód. A kulcs a diák neve, az érték az eddig megszerzett pontok összege. A végén ezt az adatszerkezetet is csökkenő sorrendbe rendezi az értékek (`Value`) alapján.
- **e - i. Szűrések és statisztikák LINQ segítségével:** * A `Where` és `Count` metódusok szűrik ki azokat, akik csak 1 versenyen indultak (jelen esetben ez a lista üres lesz, hiszen mindenki mindenhol indult).
 - Az `All` (mindenhol teljesül) és `Any` (legalább egy helyen teljesül) logikai operátorokkal dől el, kik érték el az 50 pontos minimális határt.
 - A `Take(3)` paranccsal választja ki a kód versenyenként az első három helyezettet.

- **j - m. Komplex logikai vizsgálatok:** A holtversenyek és az összetett győztes teljesítményének vizsgálata. A program a Max() függvénnyel keresi meg a legnagyobb pontszámot, majd a Count() segítségével megnézi, hány diák érte el pontosan ezt a pontszámot. Ha egynél több, akkor holtversenyt regisztrál (bool változóval).

3. A teljes forráskód

C#

using System;

using System.Collections.Generic;

using System.Linq;

namespace VersenySzimulacio

{

class Program

{

static void Main(string[] args)

{

Random rnd = new Random();

List<string> diakNevek = new List<string> { "Diak1", "Diak2", "Diak3", "Diak4",
"Diak5", "Diak6", "Diak7", "Diak8", "Diak9", "Diak10"};

// Versenyek pontszámait és eredményeit tároló lista

List<List<(string, int)>> versenyek = new List<List<(string, int)>>();

// Adatok generálása

for (int i = 0; i < 8; i++)

{

List<(string, int)> verseny = new List<(string, int)>();

Console.WriteLine(\$"Verseny {i + 1}");

for (int j = 0; j < 10; j++)

{

int pontszam = rnd.Next(0, 101); // véletlenszerű pontszám generálása

```

        verseny.Add((diakNevek[j], pontszám));

        Console.WriteLine($"{diakNevek[j]}: {pontszám}");
    }

    versenyek.Add(verseny);

    Console.WriteLine();
}

// a. Egyéni versenyek rangsora
Console.WriteLine("Egyéni versenyek rangsora:");
for (int i = 0; i < 8; i++)
{
    Console.WriteLine($"Verseny {i + 1} rangsora:");
    var egyeniRangsor = versenyek[i].OrderByDescending(x => x.Item2);
    foreach (var diak in egyeniRangsor)
    {
        Console.WriteLine($"{diak.Item1}: {diak.Item2} pont");
    }
    Console.WriteLine();
}

// b. Összetett versenyben értékelendő tanulók száma
int osszetettVersenyErtekelendoTanulokSzama = 0;
foreach (var verseny in versenyek)
{
    if (verseny.Count == 10)
    {
        osszetettVersenyErtekelendoTanulokSzama += 10;
    }
}

```

```
Console.WriteLine($"Összetett versenyben értékelendő tanulók száma:  
{osszetettVersenyErtekelendoTanulokSzama}");
```

```
// c. Összetett versenyben értékelendő tanulók neve
```

```
List<string> osszetettVersenyErtekelendoTanulokNeve = new List<string>();
```

```
foreach (var verseny in versenyek)
```

```
{
```

```
    if (verseny.Count == 10)
```

```
    {
```

```
        foreach (var diak in verseny)
```

```
        {
```

```
            osszetettVersenyErtekelendoTanulokNeve.Add(diak.Item1);
```

```
        }
```

```
    }
```

```
}
```

```
Console.WriteLine("Összetett versenyben értékelendő tanulók neve:");
```

```
foreach (var diak in osszetettVersenyErtekelendoTanulokNeve.Distinct())
```

```
{
```

```
    Console.WriteLine(diak);
```

```
}
```

```
// d. Összetett verseny rangsora
```

```
Console.WriteLine("Összetett verseny rangsora:");
```

```
Dictionary<string, int> osszetettVersenyPontszamok = new Dictionary<string, int>();
```

```
foreach (var verseny in versenyek)
```

```
{
```

```
    if (verseny.Count == 10)
```

```
    {
```

```
        foreach (var diak in verseny)
```

```

    {
        if (!osszetettVersenyPontszamok.ContainsKey(diak.Item1))
            osszetettVersenyPontszamok.Add(diak.Item1, 0);

        osszetettVersenyPontszamok[diak.Item1] += diak.Item2;
    }
}

var osszetettRangsor = osszetettVersenyPontszamok.OrderByDescending(x =>
x.Value);

foreach (var diak in osszetettRangsor)
{
    Console.WriteLine($"{diak.Key}: {diak.Value} pont");
}

// e. Versenyenként a tanulók névsora
Console.WriteLine("Versenyenként a tanulók névsora:");
for (int i = 0; i < 8; i++)
{
    Console.WriteLine($"Verseny {i + 1} tanulói:");
    foreach (var diak in versenyek[i])
    {
        Console.WriteLine(diak.Item1);
    }
    Console.WriteLine();
}

// f. Akik csak 1 versenyen indultak

```

```

Console.WriteLine("Akik csak 1 versenyen indultak:");

var csakEgyVersenyenIndultak = diakNevek.Where(diak => versenyek.Count(verseny
=> verseny.Any(x => x.Item1 == diak)) == 1);

foreach (var diak in csakEgyVersenyenIndultak)
{
    Console.WriteLine(diak);
}

// g. Akik elérték a versenyenkénti minimális pontszámot
Console.WriteLine("Akik elérték a versenyenkénti minimális pontszámot:");

var minimalisPontszam = 50;

var elertekMinimalisPontszamot = diakNevek.Where(diak => versenyek.All(verseny
=> verseny.FirstOrDefault(x => x.Item1 == diak).Item2 >= minimalisPontszam));

foreach (var diak in elertekMinimalisPontszamot)
{
    Console.WriteLine(diak);
}

// h. Akik nem érték el a versenyenkénti minimális pontszámot
Console.WriteLine("Akik nem érték el a versenyenkénti minimális pontszámot:");

var nemErtekElMinimalisPontszamot = diakNevek.Where(diak =>
versenyek.Any(verseny => verseny.FirstOrDefault(x => x.Item1 == diak).Item2 <
minimalisPontszam));

foreach (var diak in nemErtekElMinimalisPontszamot)
{
    Console.WriteLine(diak);
}

// i. Versenyenként az első 3 helyezett
Console.WriteLine("Versenyenként az első 3 helyezett:");

```

```

for (int i = 0; i < 8; i++)
{
    Console.WriteLine($"Verseny {i + 1} első 3 helyezettje:");
    var helyezettek = versenyek[i].OrderByDescending(x => x.Item2).Take(3);
    foreach (var diak in helyezettek)
    {
        Console.WriteLine($"{diak.Item1}: {diak.Item2} pont");
    }
    Console.WriteLine();
}

// j. Összetett verseny győztese, minden számban győztes volt-e
bool mindenSzambanGyoztes = true;
var osszetettGyoztes = osszetettRangsor.First().Key;
foreach (var verseny in versenyek)
{
    if (!verseny.Any(x => x.Item1 == osszetettGyoztes))
    {
        mindenSzambanGyoztes = false;
        break;
    }
}

Console.WriteLine($"Az összetett verseny győztese, minden számban győztes volt-e:
{mindenSzambanGyoztes}");

// k. Volt-e valamelyik versenyben holtverseny
bool voltHoltverseny = false;
foreach (var verseny in versenyek)
{

```



```

var maxPontszam = verseny.Max(x => x.Item2);
var legtöbbPontszammal = verseny.Count(x => x.Item2 == maxPontszam);
if (legtobbPontszammal > 1)
{
    voltHoltverseny = true;
    break;
}
}
Console.WriteLine($"Volt-e valamelyik versenyben holtverseny: {voltHoltverseny}");

// 1. Az összetett verseny győztese holtversenyben győzött-e
bool holtversenybenGyozott = false;
foreach (var verseny in versenyek)
{
    var maxPontszam = verseny.Max(x => x.Item2);
    var legtöbbPontszammal = verseny.Count(x => x.Item2 == maxPontszam);
    if (legtobbPontszammal > 1 && verseny.Any(x => x.Item1 == osszetettGyoztes))
    {
        holtversenybenGyozott = true;
        break;
    }
}
Console.WriteLine($"Az összetett verseny győztese holtversenyben győzött-e:
{holtversenybenGyozott}");

// m. Volt-e olyan tanuló, aki minden versenyen elindult, de nem érte el a minimális
pontszámot
bool mindenVersenyenElindultDeNemErteEl = false;
foreach (var diak in diakNevek)
{

```

```

bool mindenVersenyenElindult = true;
foreach (var verseny in versenyek)
{
    if (!verseny.Any(x => x.Item1 == diak))
    {
        mindenVersenyenElindult = false;
        break;
    }
}

if (mindenVersenyenElindult && !versenyek.All(verseny =>
verseny.FirstOrDefault(x => x.Item1 == diak).Item2 >= minimalisPontszam))
{
    mindenVersenyenElindultDeNemErteEl = true;
    break;
}
}

Console.WriteLine($"Volt-e olyan tanuló, aki minden versenyen elindult, de nem érte
el a minimális pontszámot: {mindenVersenyenElindultDeNemErteEl}");

Console.ReadKey();
}
}
}

```

4. A program futtatása és eredményei

Az alábbi képeken látható a program sikeres lefutása, ahol a konzolablak megjeleníti a generált eredményeket és az elvégzett statisztikai számításokat.

```
Kijelölés D:\Portfóliók\levelente23\Programozás alapjai\Iskolai verseny_44\Iskolai verseny_42\bin\Debug\net8.0\Iskolai verseny_42.exe
Verseny 1
Diak1: 37
Diak2: 88
Diak3: 33
Diak4: 39
Diak5: 30
Diak6: 57
Diak7: 39
Diak8: 78
Diak9: 98
Diak10: 37

Verseny 2
Diak1: 32
Diak2: 68
Diak3: 2
Diak4: 64
Diak5: 81
Diak6: 6
Diak7: 100
Diak8: 36
Diak9: 72
Diak10: 82

Verseny 3
Diak1: 72
Diak2: 64
Diak3: 72
Diak4: 74
Diak5: 77
```

(A fenti képen a program által generált nyers versenyeredmények láthatóak.)

```
Kijelölés D:\Portfóliók\levelente23\Programozás alapjai\Iskolai verseny_44\Iskolai verseny_42\bin\Debug\net8.0\Iskolai verseny_42.exe
Verseny 4 első 3 helyezettje:
Diak8: 71 pont
Diak10: 69 pont
Diak2: 55 pont

Verseny 5 első 3 helyezettje:
Diak1: 80 pont
Diak10: 80 pont
Diak8: 72 pont

Verseny 6 első 3 helyezettje:
Diak6: 98 pont
Diak4: 97 pont
Diak1: 94 pont

Verseny 7 első 3 helyezettje:
Diak3: 88 pont
Diak1: 86 pont
Diak2: 76 pont

Verseny 8 első 3 helyezettje:
Diak4: 93 pont
Diak5: 84 pont
Diak10: 84 pont

Az összetett verseny győztese, minden számban győztes volt-e: True
Volt-e valamelyik versenyben holtverseny: True
Az összetett verseny győztese holtversenyben győzött-e: True
Volt-e olyan tanuló, aki minden versenyen elindult, de nem érte el a minimális pontszámot: True
```

(Ezen a képen az összesített eredmények, a szűrések és az összetett rangsor látható.)

5. Önreflexió és tapasztalatok

A projekt elkészítése során sokat fejlődött az algoritmikus gondolkodásom és a problémamegoldó képességem.

Kihívások és nehézségek: A legnagyobb fejtörést az adatszerkezet kiválasztása és összeállítása jelentette. A listákba ágyazott listák és a Tuple-ök (név és pontszám párosok) együttes használata eleinte bonyolultnak tűnt, de a munka során rájöttem, hogy ez a leghatékonyabb módja egy ilyen többfordulós, összetett eredménylista tárolásának. Szintén kihívást jelentettek a bonyolultabb LINQ lekérdezések (például a holtversenyek vizsgálata, vagy annak megállapítása, hogy az összetett győztes minden számban nyert-e), ahol nagyon pontosan kellett megfogalmazni a logikai feltételeket (Any, All, Count, Max együttes használata).

Fejlődés és tanulságok: A feladat végére sokkal magabiztosabban kezdtem el használni a LINQ metódusokat. Rájöttem, hogy ezek a beépített függvények rengeteg sornyi hagyományos for ciklust és beágyazott if elágazást tudnak megspórolni, ezáltal a kód sokkal rövidebb, átláthatóbb és "tisztább" lesz. Megtanultam a Dictionary (szótár) típus hasznosságát is, ami az összetett pontszámok tárolásánál bizonyult tökéletes megoldásnak.

Továbbfejlesztési lehetőségek: Ha a jövőben továbbfejleszteném az alkalmazást, a tanulók neveit és a korábbi pontszámokat egy külső állományból (például .csv vagy .txt fájlból) olvastatnám be a konzolos adatgenerálás helyett. Emellett a projekt egy egyszerű grafikus felhasználói felülettel (GUI - pl. WPF vagy Windows Forms) is kiegészíthető lenne a jövőben, ami még vizuálisabbá tenné a végeredményt.

Összességében nagyon hasznosnak és tanulságosnak ítélem a munkát, mert egy komplex, egybefüggő feladaton keresztül tudtam bizonyítani és gyakorolni a programozási alapok elsajátítását.